

UNIVERSIDAD NACIONAL DE RAFAELA

Ingeniería en Computación

Comunicación de Datos — 1er Cuatrimestre 2026

TRABAJO PRÁCTICO N° 3 — INTEGRADOR

**Sistema de Control RGB con Comunicación Inalámbrica, Infrarroja y
Verificación de Integridad por CRC**

Autores:

Bevilacqua, Francisco

Clement, Sebastián

Docentes:

Mg. Ing. Martín Pico — Ing. Milton Pozzo

16 de junio 2026

1. Introducción

El presente informe documenta el diseño, desarrollo e implementación del Trabajo Práctico N° 3 Integrador de la asignatura Comunicación de Datos, correspondiente al primer cuatrimestre del año 2026. El trabajo constituye la instancia final de la unidad curricular, integrando los conceptos y sistemas desarrollados en los trabajos prácticos anteriores bajo un marco de aplicación domótica real.

El sistema diseñado permite el control remoto de un actuador lumínico RGB —un diodo LED tricolor— desde una interfaz gráfica de usuario ejecutada en cualquier dispositivo con navegador web moderno (teléfono celular, computadora personal o tableta). El sistema emplea tres nodos funcionales diferenciados, denominados Dispositivo 1 (D1), Dispositivo 2 (D2) y Dispositivo 3 (D3), que se comunican mediante tecnologías heterogéneas: infrarrojo (IR) en el enlace D1-D2, y Wi-Fi con WebSocket en el enlace D2-D3.

Una característica central y diferencial de la presente implementación respecto a iteraciones previas es la incorporación de verificación de integridad de datos mediante Comprobación de Redundancia Cíclica (CRC) en ambos enlaces de comunicación. En el enlace IR se utiliza CRC-8/MAXIM (polinomio 0x31), mientras que en el enlace WebSocket se emplea CRC-16/IBM (polinomio 0x8005). Esta incorporación responde a los requisitos explícitos de la consigna del TP3 y constituye el aporte técnico más significativo respecto a la versión anterior.

El sistema fue verificado en condiciones operativas reales, demostrando un comportamiento robusto: transmisión y confirmación de colores, persistencia de estado ante cortes de energía, detección bidireccional de desconexiones, y manejo correcto de errores de CRC tanto en la interfaz web como en el enlace infrarrojo.

1.1 Objetivos del Trabajo

- Implementar un sistema distribuido de tres nodos para el control cromático de un LED RGB.
- Establecer comunicación inalámbrica Wi-Fi ad hoc (modo Access Point) entre D2 y D3, sin dependencia de infraestructura de red externa ni de Internet.
- Implementar el protocolo WebSocket (RFC 6455) para comunicación full-duplex en tiempo real entre D2 y D3.
- Integrar verificación de integridad CRC-8 en el enlace infrarrojo (D1-D2) y CRC-16 en el enlace WebSocket (D2-D3).
- Garantizar coherencia de estado del sistema: persistencia en memoria no volátil, recuperación al energizarse y sincronización al reconectarse D3.
- Detectar en tiempo real la pérdida de conexión entre dispositivos y señalizarla tanto físicamente (LEDs de estado en D2) como visualmente (interfaz de D3).
- Documentar el sistema con vocabulario técnico riguroso, justificando cada decisión de diseño.

2. Marco Teórico

2.1 Comunicación Infrarroja y Protocolo NEC

La comunicación por radiación infrarroja (IR) es una forma de transmisión óptica que opera en el rango de longitudes de onda de 700 nm a 1 mm, adyacente al espectro visible por el lado del rojo. En aplicaciones de control remoto doméstico, la banda más utilizada es la cercana al infrarrojo (NIR), centrada en aproximadamente 940 nm. Esta longitud de onda es absorbida eficientemente por los fotodetectores de silicio y resulta invisible para el ojo humano, lo que la hace adecuada para aplicaciones de uso cotidiano.

La transmisión IR en sistemas de control remoto emplea modulación de portadora para distinguir la señal de mando del ruido ambiental infrarrojo (luz solar, iluminación incandescente). La técnica estándar consiste en modular la señal digital mediante una subportadora de frecuencia fija, típicamente 36 kHz, 38 kHz o 40 kHz, siendo 38 kHz la más extendida. El receptor IR (en este proyecto, el módulo KY-022) integra un fotodetector, un amplificador, un filtro de banda estrecha centrado en 38 kHz y un demodulador, entregando en su salida la señal digital demodulada con lógica invertida (activo en LOW).

El protocolo NEC (desarrollado por NEC Corporation) es uno de los estándares de control remoto IR más difundidos a nivel mundial. Define una trama de 32 bits con la siguiente estructura: pulso de arranque (9 ms en ON seguido de 4.5 ms en OFF), dirección del dispositivo (8 bits), complemento de la dirección (8 bits), comando (8 bits) y complemento del comando (8 bits). Los bits se codifican mediante una técnica de modulación por distancia entre pulsos (Pulse Distance Modulation): un bit '0' se representa con 562.5 μ s de portadora seguidos de 562.5 μ s de silencio; un bit '1' se representa con 562.5 μ s de portadora seguidos de 1687.5 μ s de silencio.

En el presente proyecto se emplea la variante NEC Raw (sendNECRaw / decodedRawData de la biblioteca IRremote), que preserva los 32 bits de la trama tal como los construye la aplicación, sin aplicar las inversiones de bits que introduce el protocolo NEC estándar en los bytes de complemento. Esto permite empaquetar libremente los componentes RGB y el byte de CRC en los 32 bits disponibles, sin restricciones del esquema dirección/complemento original del protocolo.

2.2 IEEE 802.11 — Estándar Wi-Fi

La familia de estándares IEEE 802.11 define las especificaciones de las redes de área local inalámbricas (WLAN). El ESP32 implementa las variantes 802.11b, 802.11g y 802.11n en la banda de 2.4 GHz, con velocidades de transmisión de hasta 150 Mbps (una sola cadena espacial, SISO).

En la banda de 2.4 GHz, el espectro disponible se divide en 13 canales (en Argentina) de 22 MHz de ancho cada uno, separados por 5 MHz. Los únicos canales completamente no solapados entre sí son el 1 (2.412 GHz), el 6 (2.437 GHz) y el 11 (2.462 GHz). Se seleccionó el canal 6 debido a que pertenece al conjunto de canales no superpuestos (1, 6 y 11) de la banda de 2.4 GHz. Esto reduce la probabilidad de

interferencia respecto a la utilización de canales intermedios, que presentan un elevado solapamiento espectral.

El modo de operación elegido es el modo Ad Hoc o IBSS (Infrastructure Basic Service Set) en su variante SoftAP (Software Access Point). En este modo, el ESP32 actúa simultáneamente como Access Point, gestionando la asociación de clientes (D3) y ejecutando el servidor DHCP para asignar direcciones IP automáticamente. La IP del AP es fija en 192.168.4.1 —valor por defecto del SDK Espressif— y el cliente asociado recibe la IP 192.168.4.2 por DHCP.

La seguridad de la red Wi-Fi se implementa con WPA2-Personal (Wi-Fi Protected Access 2) mediante el protocolo CCMP/AES. WPA2 utiliza el protocolo PBKDF2 (Password-Based Key Derivation Function 2) para derivar la Pairwise Master Key (PMK) a partir de la contraseña de acceso y el SSID, y luego realiza un handshake de 4 vías para generar las claves de sesión. Este esquema garantiza que el tráfico entre D3 y D2 está cifrado en la capa de enlace inalámbrico.

2.3 Protocolo WebSocket (RFC 6455)

WebSocket es un protocolo de comunicación bidireccional y full-duplex sobre TCP, estandarizado en la RFC 6455 (IETF, 2011). A diferencia de HTTP, que sigue un modelo de petición-respuesta con conexiones de corta duración, WebSocket establece un canal persistente que permite a cualquiera de los dos extremos transmitir datos en cualquier momento, sin necesidad de que el otro extremo haya realizado una solicitud previa.

El establecimiento de la conexión WebSocket se realiza mediante un proceso de negociación (handshake) sobre HTTP/1.1: el cliente envía una solicitud GET con las cabeceras 'Upgrade: websocket' y 'Connection: Upgrade', junto con una clave aleatoria en Base64 (Sec-WebSocket-Key). El servidor responde con código HTTP 101 Switching Protocols e incluye la clave de aceptación derivada de la clave del cliente. A partir de este punto, el canal TCP subyacente se reutiliza para la comunicación WebSocket, eliminando el overhead de establecimiento de conexión HTTP en cada intercambio.

Los mensajes WebSocket se transportan en frames. Cada frame contiene un encabezado de 2 a 14 bytes que especifica el opcode (tipo de mensaje: texto, binario, ping, pong, cierre), una bandera de fragmentación (FIN), la longitud del payload y, para frames del cliente al servidor, una máscara de 4 bytes aplicada mediante XOR al payload (requisito obligatorio del estándar para datos cliente-servidor). En el presente proyecto se utilizan frames de texto (opcode 0x01) con payloads en formato JSON.

El estándar define mensajes de control Ping (opcode 0x09) y Pong (opcode 0x0A) para la verificación de vivacidad de la conexión (keepalive). En la implementación del proyecto, se utiliza tanto el mecanismo de ping/pong a nivel de servidor (ESPAsyncWebServer lo ejecuta automáticamente cada 10 segundos) como un keepalive adicional a nivel de aplicación implementado en JavaScript, que detecta conexiones TCP 'zombie' que el nivel de transporte no descubriría de forma inmediata.

2.4 Verificación de Integridad: Comprobación de Redundancia Cíclica (CRC)

La Comprobación de Redundancia Cíclica (CRC, Cyclic Redundancy Check) es un esquema de detección de errores ampliamente utilizado en protocolos de comunicación digital. Su fundamento matemático es la aritmética de polinomios sobre el campo de Galois GF(2), donde la adición se realiza mediante la operación XOR (OR exclusivo) y no existe acarreo. En este contexto, un bloque de datos se interpreta como los coeficientes de un polinomio binario $M(x)$, que se divide (mediante la división polinómica en GF(2)) por un polinomio generador $G(x)$ de grado k . El residuo de esta división, de k bits, constituye el CRC o Frame Check Sequence (FCS).

El proceso de verificación en el receptor consiste en calcular el CRC sobre los datos recibidos y compararlo con el CRC transmitido. Si ambos son iguales, con alta probabilidad los datos no sufrieron corrupción. La capacidad de detección de un CRC depende del polinomio generador elegido y se caracteriza por la Distancia de Hamming (HD): un CRC con $HD=d$ garantiza la detección de todos los patrones de error de peso menor a d . Los polinomios estándares son seleccionados por sus propiedades algebraicas para maximizar la distancia de Hamming para longitudes de mensaje específicas.

La implementación por software del CRC se realiza mediante el algoritmo bit a bit: para cada bit del mensaje de entrada, se actualiza un registro de k bits aplicando XOR con el polinomio generador cuando el bit de mayor o menor peso del registro es '1' (según la convención de procesamiento MSB-first o LSB-first, respectivamente). Este algoritmo, aunque más lento que las implementaciones con tabla de búsqueda, es transparente, portable entre plataformas y de longitud reducida (aproximadamente 15 líneas de código), lo que lo hace ideal para el contexto académico de este proyecto donde la comprensión del proceso es prioritaria.

2.4.1 CRC-8/MAXIM (Dallas/1-Wire)

El polinomio CRC-8/MAXIM, también conocido como CRC-8/Dallas o CRC-8/1-Wire, fue definido por Maxim Integrated (Dallas Semiconductor) para su uso en dispositivos del bus 1-Wire. El polinomio generador es:

$$G(x) = x^8 + x^5 + x^4 + 1 \rightarrow 0x31 \text{ (representación hexadecimal)}$$

Parámetro	Valor	Descripción
Poly	0x31	Polinomio generador: $x^8 + x^5 + x^4 + 1$
Init	0xFF	Registro inicializado en 0xFF (evita que la entrada de ceros produzca CRC nulo)
RefIn	true	Procesamiento LSB-first (reflexión de entrada)
RefOut	true	Reflexión del byte de resultado
XorOut	0x00	Sin XOR final sobre el resultado
HD	4	Distancia de Hamming para mensajes de hasta 119 bits (15 bytes)

La distancia de Hamming HD=4 para el CRC-8/MAXIM garantiza la detección de todos los errores de 1, 2 y 3 bits en mensajes de hasta 119 bits (≈15 bytes). Dado que los datos útiles del enlace IR son únicamente 3 bytes (R, G, B), este polinomio proporciona una cobertura matemáticamente suficiente con probabilidad de detección prácticamente del 100% para patrones de error realistas en un canal IR de corto alcance.

La elección de Init = 0xFF en lugar del valor 0x00 es fundamental para la robustez del protocolo. Si el registro se inicializa en 0x00 y el mensaje de entrada es todo ceros (situación que puede ocurrir ante ruido eléctrico o señales espurias interpretadas como tramas nulas), el CRC resultante sería 0x00, que coincidiría con el CRC calculado por el receptor sobre los mismos datos nulos, dando una falsa validación. Con Init = 0xFF, el CRC de una trama de ceros es un valor no nulo, lo que impide que el ruido ambiental produzca falsos positivos de validación.

2.4.2 CRC-16/IBM (CRC-16/ARC)

El polinomio CRC-16/IBM, también denominado CRC-16/ARC o CRC-16/LHA, es uno de los estándares de CRC de 16 bits más utilizados en la industria. Es empleado en protocolos como USB (en variante modificada), Modbus RTU y numerosos sistemas de almacenamiento. Su polinomio generador es:

$$G(x) = x^{16} + x^{15} + x^2 + 1 \rightarrow 0x8005$$

Parámetro	Valor	Descripción
Poly	0x8005	Polinomio generador: $x^{16} + x^{15} + x^2 + 1$
Init	0x0000	Registro inicializado en cero
RefIn	true	Procesamiento LSB-first
RefOut	true	Reflexión del resultado de 16 bits
XorOut	0x0000	Sin XOR final
HD	4	Distancia de Hamming para mensajes de hasta 32.767 bits

La justificación técnica de emplear CRC-16 en el enlace WebSocket en lugar de CRC-8 es la diferencia de longitud de los mensajes. Los payloads JSON transportados entre D2 y D3 tienen una extensión de hasta aproximadamente 200-300 bytes (1.600-2.400 bits). Para esas longitudes, la distancia de Hamming del CRC-8/MAXIM se degrada a HD=2, lo que implica que solo garantiza la detección de errores de un único bit. El CRC-16/IBM, en cambio, mantiene HD=4 para mensajes de hasta 32.767 bits (4.096 bytes), proporcionando cobertura completa para todos los mensajes del sistema con margen amplio.

3. Hardware y Tecnologías Utilizadas

3.1 Microcontrolador ESP32 DevKit v1

El ESP32 (Espressif Systems) es un SoC (System on Chip) de doble núcleo basado en la arquitectura Xtensa LX6, con procesadores de 32 bits que operan a 240 MHz. Incorpora 520 KB de SRAM, 4 MB de memoria flash externa, radio Wi-Fi 802.11 b/g/n de 2.4 GHz, Bluetooth 4.2/BLE, y un extenso conjunto de periféricos de hardware (UART, SPI, I2C, I2S, ADC, DAC, LEDC, RMT, entre otros). En el presente proyecto se utilizan dos unidades ESP32 DevKit v1: una para el D1 (Esclavo IR) y otra para el D2 (Maestro IR + Servidor Wi-Fi).

Característica	Especificación
CPU	Xtensa LX6 dual-core, 240 MHz
SRAM	520 KB
Flash	4 MB (SPI externa)
Wi-Fi	802.11 b/g/n, 2.4 GHz, hasta 150 Mbps
GPIO	34 pines (18 de solo entrada en 34-39)
PWM (LEDC)	16 canales independientes, hasta 16 bits de resolución
RMT	8 canales de modulación/demodulación hardware
NVS	Almacenamiento clave-valor en flash con wear leveling
Tensión de operación	3.3 V (entradas tolerantes a 5 V seleccionadas)

3.2 Módulos Infrarrojos KY-005 y KY-022

El módulo KY-005 contiene un LED infrarrojo (emisor) de 940 nm con resistencia limitadora integrada. Opera a 3.3 V y es compatible con los GPIO del ESP32 sin circuito adaptador. El módulo KY-022 integra un fotodetector IR, un amplificador de transimpedancia, un filtro de banda estrecha centrado en 38 kHz y un demodulador con salida TTL invertida. Acepta alimentación de 2.7 V a 5.5 V, lo que lo hace compatible con los 3.3 V del ESP32. La distancia de operación efectiva en condiciones de laboratorio (sin luz ambiente intensa) es de hasta 5-8 metros, ampliamente suficiente para la aplicación.

El canal RMT (Remote Control Transceiver) del ESP32 —utilizado internamente por la biblioteca IRremote para modular la portadora de 38 kHz— es un periférico de hardware especializado que genera y decodifica señales moduladas con alta precisión temporal, sin intervención de la CPU durante la transmisión. Esto libera el núcleo de procesamiento para tareas concurrentes durante los períodos de transmisión IR.

3.3 LED RGB de Cátodo Común

El actuador principal del sistema es un LED RGB de cátodo común, que integra tres diodos emisores de luz (rojo, verde y azul) en un único encapsulado con un cátodo

compartido. Cada canal requiere una resistencia limitadora de corriente en serie con el ánodo correspondiente. Los tres canales se controlan mediante señales PWM generadas por el módulo LEDC (LED Control) del ESP32.

El módulo LEDC del ESP32 reemplaza la función `analogWrite()` del Arduino clásico. Se configura con una frecuencia de 5 kHz y una resolución de 8 bits (256 niveles de ciclo de trabajo, de 0 a 255), que corresponde directamente a los valores RGB del protocolo de comunicación. La frecuencia de 5 kHz supera ampliamente el umbral de percepción de parpadeo humano (generalmente considerado por debajo de 60 Hz), garantizando una apariencia de brillo continuo a cualquier nivel de intensidad.

3.4 LEDs de Estado (D2)

El Dispositivo 2 incorpora tres LEDs de estado que proveen retroalimentación visual inmediata sobre el estado del sistema, independientemente de la interfaz web de D3. Cada LED está conectado a un GPIO de propósito general con una resistencia limitadora de 330 Ω en serie.

LED	Color	GPIO	Significado
LED CONN	Verde	32	D3 tiene sesión WebSocket activa
LED DISC	Rojo	33	Desconexión de D3 detectada (activo cuando D3 no está conectado)
LED AP	Amarillo	14	Access Point Wi-Fi activo — sistema en operación normal

3.5 Entorno de Desarrollo: PlatformIO

El desarrollo del firmware se realizó con PlatformIO IDE, una extensión de Visual Studio Code que provee un ecosistema de compilación, flasheo y depuración para microcontroladores. PlatformIO gestiona automáticamente la descarga del toolchain Xtensa LX6, la resolución de dependencias de bibliotecas con versionado semántico, y la compilación incremental mediante SCons. El archivo `platformio.ini` de cada proyecto declara la plataforma (`espressif32`), la placa (`esp32dev`), el framework (`arduino`) y las dependencias de bibliotecas, garantizando reproducibilidad del entorno entre diferentes máquinas de desarrollo.

3.6 Bibliotecas de Software

Biblioteca	Versión	Función en el proyecto
z3t0/IRremote	^4.7.1	Protocolo NEC Raw IR: <code>IrSender.sendNECRaw()</code> , <code>IrReceiver.decode()</code>
me-no-dev/AsyncTCP	^1.1.1	Sockets TCP asíncronos sobre lwIP para el D2
me-no-dev/ESPAsyncWebServer	^1.2.3	Servidor HTTP no bloqueante + gestor de WebSocket (D2)
bblanchon/ArduinoJson	^7.2.0	Serialización/deserialización JSON (JsonDocument unificado de v7)

Preferences (ESP32 Core)	Built-in	Persistencia NVS sobre flash con wear leveling automático
WiFi (ESP32 Core)	Built-in	Modo SoftAP (WiFi.softAP()) para crear la red Wi-Fi de D2

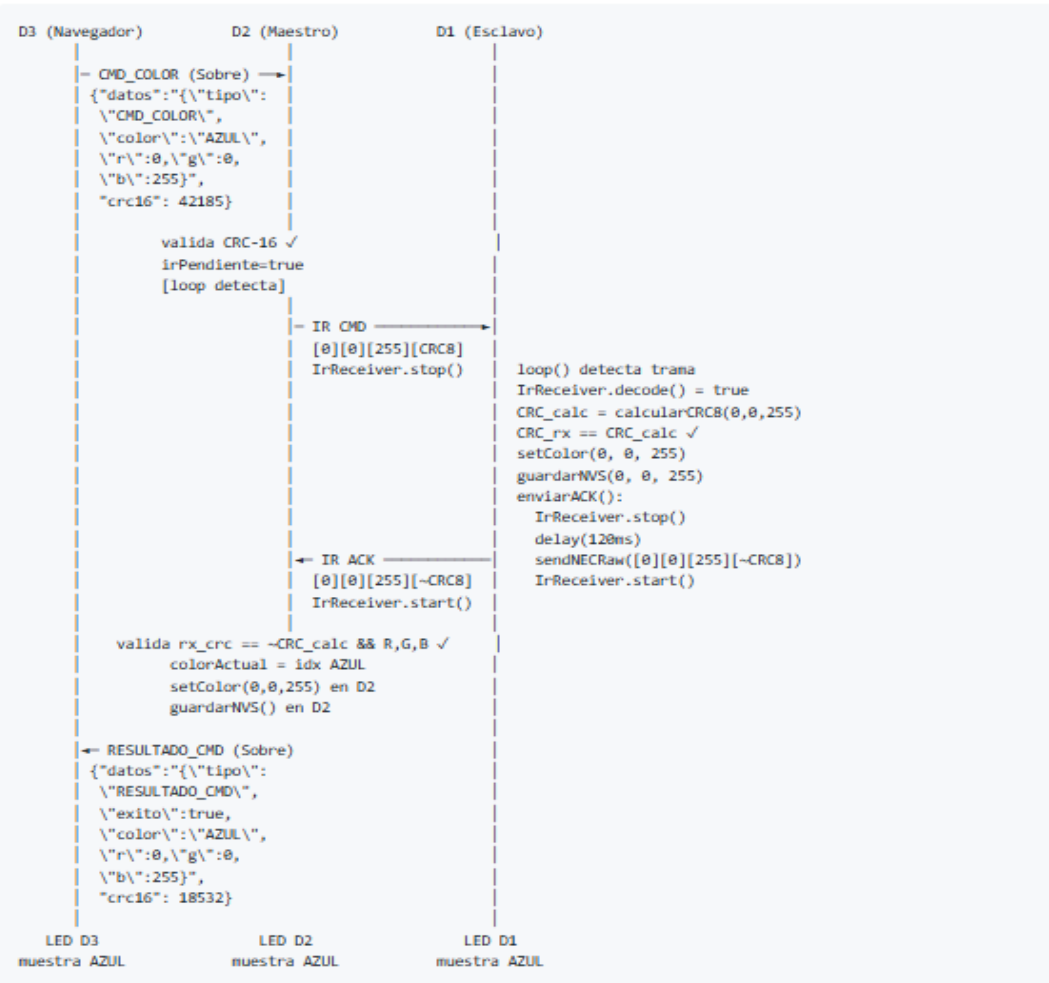
4. Arquitectura del Sistema

4.1 Topología General

El sistema adopta una arquitectura de tres nodos en cascada, donde D2 cumple el rol de nodo central y punto de convergencia entre dos dominios de comunicación distintos: el dominio inalámbrico de radiofrecuencia (Wi-Fi 2.4 GHz, hacia D3) y el dominio inalámbrico óptico (infrarrojo, hacia D1).

11. Flujo de datos completo

Secuencia completa de una operación exitosa, desde que D3 presiona un botón de color hasta que el LED de D1 cambia:



Dispositivo	Rol	Hardware	Comunicación
D1 — Esclavo IR	Actuador: controla el LED RGB físico	ESP32 DevKit v1 + KY-022 + KY-005 + LED RGB	IR NEC Raw 32 bits + CRC-8 (esclavo)
D2 — Maestro IR + Servidor	Nodo central: traduce comandos Wi-Fi a IR	ESP32 DevKit v1 + KY-005 + KY-022 + LED RGB + LEDs estado	IR NEC Raw (maestro) + Wi-Fi AP + HTTP + WebSocket
D3 — Interfaz Gráfica	Interfaz de usuario: navegador web	Cualquier dispositivo con navegador	Wi-Fi cliente + WebSocket + CRC-16 en JavaScript

		moderno (celular, PC)	
--	--	-----------------------	--

4.2 Dispositivo 1 — Esclavo IR

El D1 es el nodo actuador del sistema. Su firmware implementa un único ciclo de espera activa: permanece indefinidamente a la escucha de tramas IR provenientes del KY-022. Cuando `IrReceiver.decode()` reporta una trama completa, el firmware extrae los cuatro bytes de la trama de 32 bits, verifica la integridad mediante CRC-8 y, si el CRC es válido, aplica el color RGB al LED mediante los canales LEDC, persiste el estado en NVS y transmite una trama de ACK al D2 con los mismos valores RGB. Si el CRC no coincide, la trama se descarta silenciosamente sin ninguna acción sobre el actuador.

Al energizarse, el D1 recupera el último estado de color guardado en la NVS y lo aplica inmediatamente al LED, garantizando que un corte momentáneo de energía no requiera la intervención de D2 o D3 para restaurar el estado visual.

El D1 no posee lógica de control propia ni puede iniciar comunicaciones: su única autonomía es la recuperación de estado desde NVS en el arranque. Toda decisión de cambio de color proviene de D2.

4.3 Dispositivo 2 — Maestro IR y Servidor

El D2 es el nodo más complejo del sistema. Combina en un único firmware dos roles de comunicación radicalmente distintos: actúa como Maestro IR hacia D1 (iniciando todas las transmisiones de color) y como servidor de red hacia D3 (proveyendo la interfaz web y gestionando la sesión WebSocket).

El firmware del D2 opera sobre los dos núcleos Xtensa LX6 del ESP32 mediante el scheduler FreeRTOS: el núcleo 0 ejecuta la pila lwIP/TCP y los callbacks de `ESPAsyncWebServer` (incluyendo los eventos WebSocket), mientras que el núcleo 1 ejecuta el `loop()` de Arduino con la lógica de aplicación y el ciclo IR. Esta separación de tareas entre núcleos permite que el servidor WebSocket continúe detectando eventos de red —incluyendo desconexiones de D3— incluso durante los períodos en que el `loop()` está bloqueado ejecutando el ciclo IR (que puede durar hasta ~6.9 segundos en el peor caso con tres reintentos y dos segundos de timeout por intento).

La sincronización entre los dos contextos de ejecución (callback WS en núcleo 0 y `loop` en núcleo 1) se realiza mediante variables compartidas declaradas `volatile`, que instruyen al compilador a leer y escribir siempre desde memoria principal sin cachear los valores en registros de CPU. El patrón de acceso (un único escritor por variable: el callback WS solo escribe `irPendiente = true`; el `loop` solo lo lee y limpia) es seguro sin mutex explícito en el ESP32, porque la escritura de `bool` y `uint8_t` es atómica a nivel hardware.

4.4 Dispositivo 3 — Interfaz Gráfica de Usuario

El D3 es cualquier dispositivo con un navegador web moderno (Google Chrome, Mozilla Firefox, Safari, Microsoft Edge) con soporte de la API WebSocket nativa. En las pruebas

del proyecto se utilizó un teléfono celular y una computadora conectado a la red Wi-Fi del AP de D2.

La interfaz web se almacena íntegramente en la memoria flash del ESP32-D2 como un string literal PROGMEM de aproximadamente 15 KB, y es servida por ESPAsyncWebServer en la ruta raíz ('/') mediante el método send_P(), que lee el contenido directamente desde flash sin copiarlo a SRAM. Esta decisión es técnicamente necesaria dado que la SRAM disponible del ESP32 (520 KB) debe reservarse principalmente para las estructuras dinámicas de la pila TCP, los buffers de WebSocket y los objetos de ArduinoJson, con poco margen para almacenar la página HTML completa.

La interfaz fue implementada como una Single Page Application (SPA) mínima, sin dependencias externas, utilizando únicamente las APIs estándar del navegador: WebSocket nativo, manipulación del DOM con JavaScript, y CSS con variables custom para el sistema de diseño. La ausencia de frameworks externos o CDN es un requisito implícito del sistema, dado que D2 opera como AP aislado sin acceso a Internet.

5. Conexionado de Hardware

5.1 Dispositivo 1 — Esclavo IR

Componente / Pin	GPIO ESP32	Función
KY-022 OUT (receptor IR)	GPIO 22	Entrada de señal demodulada (activo LOW)
KY-005 S (emisor IR)	GPIO 23	Salida PWM modulada en 38 kHz (canal RMT)
LED RGB — Rojo	GPIO 25 (LEDC CH3)	Canal PWM rojo, cátodo común a GND
LED RGB — Verde	GPIO 26 (LEDC CH1)	Canal PWM verde
LED RGB — Azul	GPIO 27 (LEDC CH2)	Canal PWM azul
KY-022 + KY-005 VCC	3.3 V	Alimentación de módulos IR
GND	GND	Referencia común a todos los módulos

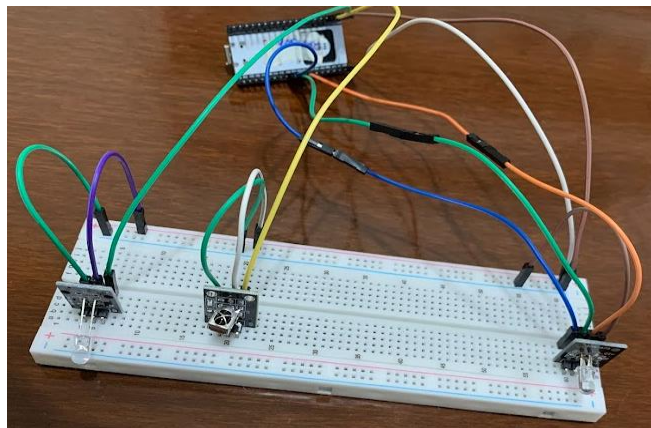


Foto: circuito físico del Dispositivo 1 – Esclavo.

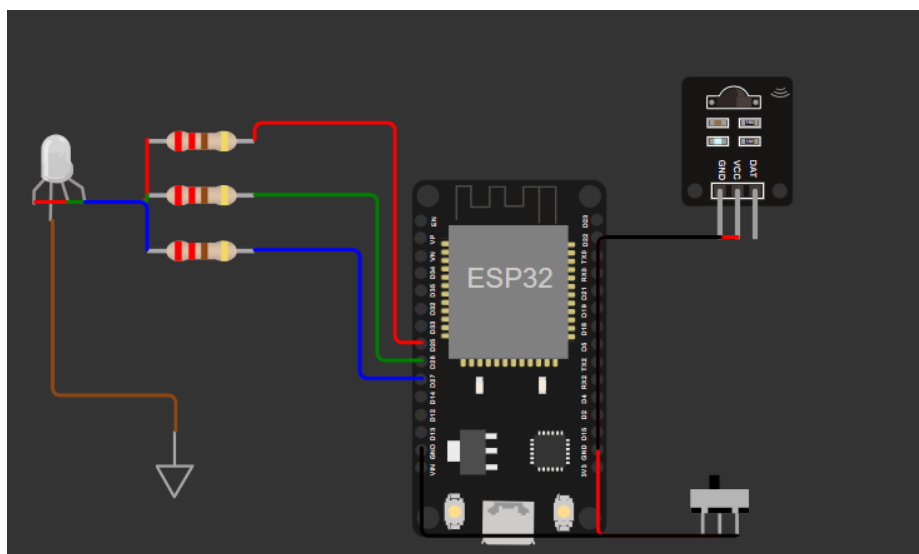


Foto: diagrama eléctrico del Dispositivo 1-Esclavo. Software: WOKWI

5.2 Dispositivo 2 — Maestro IR + Servidor

Componente / Pin	GPIO ESP32	Función
KY-005 S (emisor IR)	GPIO 4	Salida PWM modulada 38 kHz para D1
KY-022 OUT (receptor IR)	GPIO 5	Entrada para ACK de D1
LED RGB — Rojo	GPIO 25 (LEDC CH3)	Espejo visual del color actual del sistema
LED RGB — Verde	GPIO 26 (LEDC CH1)	Canal PWM verde
LED RGB — Azul	GPIO 27 (LEDC CH2)	Canal PWM azul
LED CONN (verde)	GPIO 32	Indicador: D3 conectado con WebSocket activa
LED DISC (rojo)	GPIO 33	Indicador: D3 desconectado
LED AP (amarillo)	GPIO 14	Indicador: Access Point Wi-Fi activo
KY-022 + KY-005 VCC	3.3 V	Alimentación de módulos IR
Resistencias LED RGB	$220\ \Omega \times 3$	Limitación de corriente canales RGB
Resistencias LEDs estado	$330\ \Omega \times 3$	Limitación de corriente LEDs CONN/DISC/AP

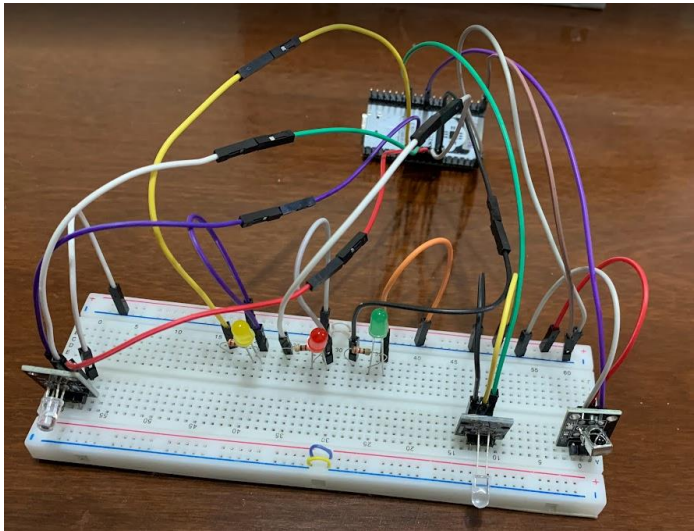


Foto: circuito físico del Dispositivo 2- Maestro.

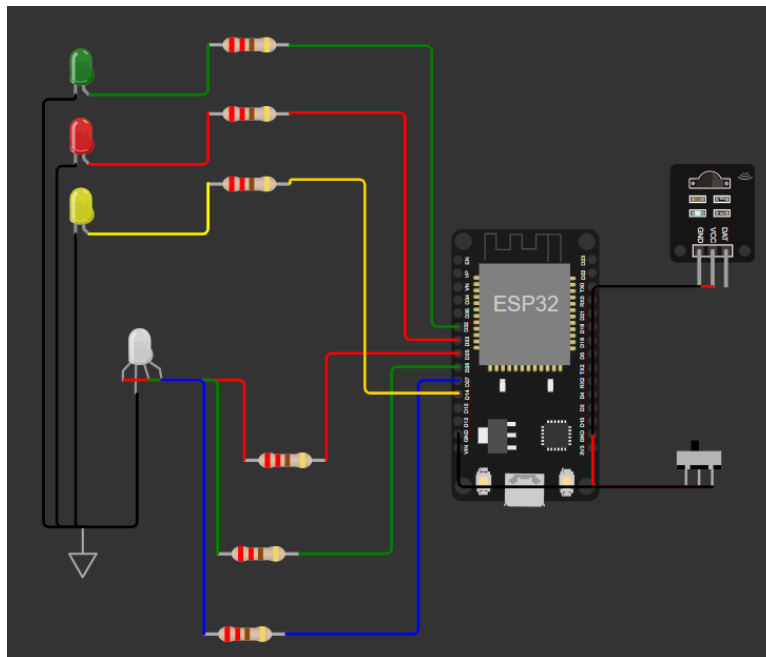


Foto: diagrama eléctrico del Dispositivo 2-Maestro. Software: WOKWI

5.3 Criterios de Selección de Pines GPIO

La selección de los pines GPIO responde a las restricciones específicas del ESP32 DevKit v1. Los siguientes GPIO presentan restricciones en el arranque y deben evitarse para funciones de aplicación: GPIO 0 (selector de modo boot, debe estar en HIGH para arranque normal), GPIO 2 (debe estar en LOW durante la programación flash), GPIO 6-11 (conectados al bus SPI de la flash interna, nunca disponibles para uso de aplicación), GPIO 12 (MTDI, configura el voltaje de la flash en el boot) y GPIO 15 (MTDO, silencia el log de boot si está en LOW). Los GPIO 34-39 son de solo entrada sin resistencias de pull-up/pull-down internas.

El canal LEDC 3 se asigna al canal rojo del LED RGB en lugar del canal 0 porque la biblioteca IRremote reserva internamente el canal LEDC 0 (y el canal RMT 0) para la generación de la portadora de 38 kHz en el KY-005. Usar el canal 0 para el LED produjo conflictos con IRremote que se manifestaron como interrupciones en la transmisión IR y/o en el PWM del LED.

6. Implementación de Software

6.1 Firmware del Dispositivo 1 — Esclavo IR

6.1.1 Estructura y Flujo Principal

El firmware del D1 implementa una máquina de estados simple basada en el ciclo `setup()/loop()` del framework Arduino. La función `setup()` inicializa los periféricos en el siguiente orden: puerto serie para depuración, canales LEDC para el LED RGB, receptor IR, y recuperación del estado desde NVS. La función `loop()` implementa el ciclo de recepción y respuesta IR.

El `loop()` verifica en cada iteración si `IrReceiver.decode()` retorna una trama completa. Si no hay datos, retorna inmediatamente (cede CPU al scheduler de FreeRTOS). Si hay datos, aplica el siguiente protocolo:

1. Filtrar repeticiones NEC automáticas (flag `IRDATA_FLAGS_IS_REPEAT`).
2. Leer `decodedRawData` antes de llamar a `resume()` (`resume()` limpia el buffer y haría el dato ilegible).
3. Llamar a `IrReceiver.resume()` inmediatamente para liberar el buffer y habilitar la próxima recepción.
4. Extraer los cuatro bytes de la trama de 32 bits: R (bits 31-24), G (bits 23-16), B (bits 15-8), CRC (bits 7-0).
5. Calcular `calcularCRC8(r, g, b)` y comparar con el CRC recibido. Si no coinciden, descartar silenciosamente.
6. Si el CRC es válido: `setColor(r, g, b)`, `guardarNVS(r, g, b)`, `enviarACK(r, g, b)`.

6.1.2 Gestión Half-Duplex del Canal IR

El canal IR del D1 es half-duplex: el KY-005 y el KY-022 comparten el medio óptico y no pueden operar simultáneamente. La función `enviarACK()` gestiona esta coordinación mediante el siguiente protocolo:

1. **`IrReceiver.stop()`** — detiene el receptor para evitar que el KY-022 capture el eco de la propia transmisión.
2. **`delay(120 ms)`** — tiempo de guardia para que el D2 complete su secuencia `stop/transmit/start` y tenga el receptor activo antes de que llegue el ACK.
3. **`IrSender.sendNECRaw(trama, 0)`** — transmite la trama ACK con la misma estructura de 32 bits `[R][G][B][~CRC]`.
4. **`IrReceiver.start()`** — reactiva el receptor para la próxima trama de comando.

6.2 Firmware del Dispositivo 2 — Maestro IR y Servidor

6.2.1 Ciclo IR con Reintentos

La función `ejecutarCicloIR()` implementa la lógica de envío con reintentos: intenta transmitir la trama IR hasta `MAX_REINTENTOS=3` veces. Cada intento llama a `enviarColorIR(r, g, b)`, que transmite la trama y espera el ACK por hasta `TIMEOUT_ACK_MS=2000 ms`. Si el ACK es válido (CRC correcto y datos coincidentes), actualiza el estado del sistema, aplica el color al LED del D2, persiste en NVS y notifica

a D3. Si ningún intento obtiene ACK, el estado no cambia y D3 recibe una notificación de fallo (RESULTADO_CMD con éxito=false).

6.2.2 Servidor WebSocket y Procesamiento de Mensajes

El callback onWsEvent() es el punto de entrada de toda la comunicación con D3. Procesa cuatro tipos de eventos:

- WS_EVT_CONNECT: registra el cliente, actualiza los LEDs de estado (CONN encendido, DISC apagado) y envía el ESTADO_ACTUAL con CRC-16 al cliente recién conectado.
- WS_EVT_DISCONNECT: actualiza los LEDs (CONN apagado, DISC encendido).
- WS_EVT_DATA: verifica que el frame sea de texto y no fragmentado, luego llama a procesarMensajeWS() para parsear el JSON y actuar según el tipo de mensaje.
- WS_EVT_ERROR: registra el error en el log de depuración.

La función procesarMensajeWS() implementa la validación CRC-16 como primera operación sobre cualquier mensaje recibido. Si el CRC no es válido, responde con CRC_ERROR y descarta el mensaje sin modificar el estado del sistema. Si el CRC es válido, procesa el tipo de mensaje y activa irPendiente=true para que el loop() ejecute el ciclo IR.

6.2.3 Desconexión Voluntaria con Handshake

La desconexión voluntaria de D3 (activada por el botón 'Desconectar' en la interfaz web) implementa un handshake de cierre a nivel de aplicación para garantizar la actualización inmediata de los LEDs de estado en D2. El protocolo de dos pasos es:

```
D3 → D2: { "tipo": "CMD_DESCONECTAR", "crc16": XXXX }
D2 → D3: { "tipo": "ACK_DESCONECTAR", "crc16": XXXX }
D3: ws.close(1000, "Cierre voluntario")
D2: WS_EVT_DISCONNECT → setLedConn(false), setLedDisc(true)
```

Este mecanismo resuelve una limitación de la pila TCP asíncrona: si D2 cerrara el socket unilateralmente con client->close(), el handshake TCP FIN/ACK podría quedar pendiente por varios segundos en el contexto de la pila lwIP antes de que el evento WS_EVT_DISCONNECT se dispare. Con el cierre iniciado desde el cliente, la secuencia TCP se completa de forma ordenada y el evento de desconexión es procesado por el Core 0 en tiempo mínimo.

6.3 Interfaz Gráfica — Dispositivo 3

La interfaz web fue diseñada como una SPA dark-theme con tema de terminal, implementada en HTML5, CSS3 y JavaScript vanilla sin dependencias externas. Los elementos principales son:

- Badge de estado de conexión: muestra 'Conectado', 'Desconectado' o 'Conectando...' con codificación de color (verde/rojo/amarillo).

- Preview de color: bloque visual que refleja el color actual del LED de D1, incluyendo el nombre del color y los valores RGB numéricos.
- Botones de encendido/apagado: controlan el estado on/off del LED sin cambiar el color seleccionado.
- Grid de colores: 12 botones de selección de color (paleta ampliada respecto al TP2), con resaltado del color activo.
- Banner de notificaciones: muestra el resultado de cada comando (éxito/fallo) y errores de CRC.
- Botón de conexión/desconexión: permite conectar o desconectar voluntariamente la sesión WebSocket.

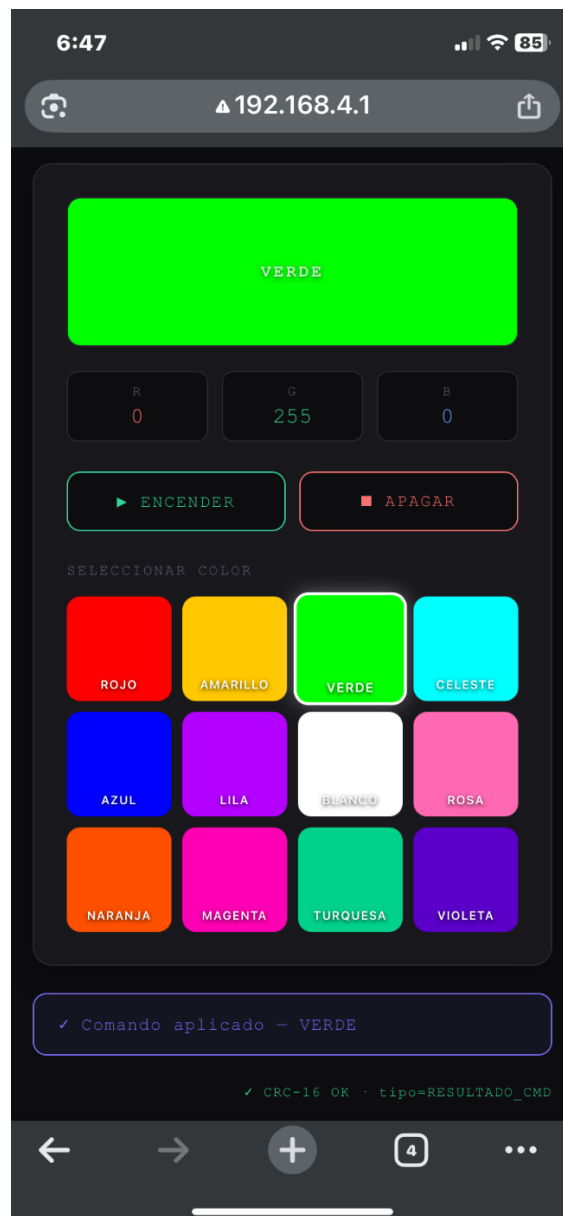


Foto: interfaz grafica que se muestra en el dispositivo 3 que se conecta con D2.

La lógica de keepalive en JavaScript envía un mensaje PING al D2 cada 5 segundos y espera un PONG en los siguientes 3 segundos. Si el PONG no llega, el WebSocket se cierra forzosamente y la interfaz muestra el aviso de desconexión. Este mecanismo complementa el ping/pong nativo del protocolo WebSocket gestionado por ESPAsyncWebServer.

La implementación del CRC-16 en JavaScript utiliza el operador de desplazamiento lógico sin signo ($\gg$) en lugar del desplazamiento aritmético ($\gg$) para simular un registro de 16 bits sin signo, dado que JavaScript no tiene tipos enteros sin signo nativos. El resultado se enmascara con $\& 0xFFFF$ para descartar bits superiores al bit 15, garantizando que la función produce el mismo resultado que su equivalente en C++ del D2 para cualquier entrada.

7. Implementación Detallada de CRC

7.1 CRC-8 en el Enlace Infrarrojo (D1 ↔ D2)

7.1.1 Estructura de la Trama IR con CRC-8

La trama infrarroja de 32 bits transmitida sobre NEC Raw tiene la siguiente estructura en la implementación final:

Trama CMD de D2 a D1:

```
Bits 31-24: R (0-255) | Bits 23-16: G (0-255) | Bits 15-8: B
                  (0-255) | Bits 7-0: CRC-8
```

Trama ACK de D1 a D2:

```
Bits 31-24: R (0-255) | Bits 23-16: G (0-255) | Bits 15-8: B
(0-255) | Bits 7-0: ~CRC-8(CRC_8_NORMAL invertido lógicamente)
```

Una consecuencia de esta estructura es que la comunicación exige diferenciar el emisor original para evitar colisiones. Por ello, se implementó una Asimetría de Protocolo en la Capa de Enlace: el Maestro (D2) envía la trama de comando firmada con el CRC-8/MAXIM normal. El Esclavo (D1), al responder, envía el ACK invirtiendo a nivel de bits la firma matemática del CRC (Operador NOT, ~CRC).

D2 valida el ACK calculando el CRC local y verificando que la firma recibida sea exactamente el complemento a 1 (~CRC). Si la firma coincide con el CRC normal, D2 identifica la señal como un 'eco óptico' de su propia transmisión y la descarta, garantizando el determinismo del enlace half-duplex."

D2 valida el ACK verificando que el CRC sea correcto y que los valores R, G, B coincidan con los enviados.

7.1.2 Algoritmo CRC-8 — Código Fuente Completo

La función `calcularCRC8()` implementa el estándar CRC-8/MAXIM mediante el algoritmo bit a bit. La misma función es utilizada tanto en D1 como en D2:

```
uint8_t calcularCRC8(uint8_t r, uint8_t g, uint8_t b) {
    uint8_t crc = 0xFF;           // Init = 0xFF
    uint8_t datos[3] = { r, g, b };

    for (uint8_t i = 0; i < 3; i++) {
        crc ^= datos[i];          // XOR con byte de datos

        for (uint8_t bit = 0; bit < 8; bit++) {
            if (crc & 0x01) {      // LSB = 1 (RefIn: LSB-first)
                crc = (crc >> 1) ^ CRC8_POLY; // desplazar y XOR con
polinomio
            } else {
                crc >>= 1;         // solo desplazar
            }
        }
    }
}
```

```

    }
    return crc;
}

```

El algoritmo procesa un byte a la vez: primero aplica XOR del byte de datos al registro CRC, luego itera bit a bit: si el LSB del registro es 1, desplaza a la derecha y aplica XOR con el polinomio (0x31); si el LSB es 0, solo desplaza. El procesamiento LSB-first (RefIn=true) es equivalente a procesar el mensaje bit a bit desde el bit menos significativo, conforme al estándar CRC-8/MAXIM.

7.1.3 Ejemplo de Cálculo: Color AMARILLO (R=255, G=200, B=0)

Para ilustrar el funcionamiento del algoritmo, se presenta el cálculo del CRC-8 para el color AMARILLO (R=255, G=200, B=0):

```
Init:   crc = 0xFF = 11111111b
```

```
Byte 0 (R=255=0xFF):
```

```
  crc XOR 0xFF = 0xFF XOR 0xFF = 0x00
```

```
  8 iteraciones sobre 0x00 → crc = 0x00 (todos bits 0, solo desplaza)
```

```
Byte 1 (G=200=0xC8):
```

```
  crc XOR 0xC8 = 0x00 XOR 0xC8 = 0xC8 = 11001000b
```

```
  Bit 0: LSB=0 → crc = 0x64 (01100100b)
```

```
  Bit 1: LSB=0 → crc = 0x32 (00110010b)
```

```
  Bit 2: LSB=0 → crc = 0x19 (00011001b)
```

```
  Bit 3: LSB=1 → crc = (0x19>>1) XOR 0x31 = 0x0C XOR 0x31 = 0x3D
```

```
  ... (continuación de las 8 iteraciones)
```

```
  Resultado tras byte 1: crc = valor intermedio
```

```
Byte 2 (B=0):
```

```
  crc XOR 0x00 = crc (sin cambio)
```

```
  ... 8 iteraciones
```

```
Resultado final: CRC8(255, 200, 0) → valor único no nulo
```

```
Trama IR completa: 0xFFC800XX (donde XX es el CRC calculado)
```

El resultado exacto del CRC para AMARILLO puede verificarse con calculadoras en línea de CRC-8/MAXIM. La implementación fue validada comparando los resultados del cálculo en C++ (D2) con los valores de referencia del estándar antes de la integración en el sistema.

7.1.4 Justificación de Init = 0xFF

La elección del valor de inicialización Init = 0xFF es una decisión de diseño crítica para la robustez del protocolo IR. El canal infrarrojo está sujeto a ruido ambiental (reflexiones de luz solar, iluminación fluorescente o LED con componente IR, ruido eléctrico en el receptor). En condiciones de ruido, el KY-022 puede reportar tramas espurias de valor 0x00000000.

Si el registro CRC se inicializara en 0x00 (Init = 0x00), el CRC calculado sobre una entrada de tres bytes nulos sería:

```

CRC8_Init0x00(0, 0, 0):
  crc = 0x00; crc XOR 0 = 0x00; 8 bits = 0 → crc = 0x00
  (para los tres bytes) → CRC = 0x00
  Trama espuria: 0x00000000 → byte CRC = 0x00 → ¡validación falsa!

```

Con Init = 0xFF, el CRC de tres bytes nulos no es cero, y la trama espuria 0x00000000 produce una discrepancia entre el CRC transmitido (0x00) y el CRC calculado por el receptor, resultando en el rechazo correcto de la trama.

Este problema fue identificado durante las pruebas del sistema y documentado como uno de los problemas encontrados en el desarrollo. La modificación del valor de inicialización eliminó completamente el bucle de retroalimentación infinita producido por el 'eco óptico' (D1 captando el eco de su propia transmisión ACK y ejecutando otro ACK).

7.2 CRC-16 en el Enlace WebSocket (D2 ↔ D3)

7.2.1 Estructura del Protocolo de Mensajes con CRC-16

El protocolo WebSocket entre D2 y D3 emplea el patrón de 'sobre' (Envelope Pattern) para la verificación de integridad CRC-16. Este patrón fue adoptado para resolver el problema de indeterminismo estructural del formato JSON: el estándar JSON (ECMA-404) no garantiza el orden de las claves en la representación serializada. Si el CRC se calculara sobre el objeto JSON incluyendo el propio campo 'crc16', la deserialización y re-serIALIZACIÓN en cualquiera de los dos extremos podría alterar el orden de las claves, generando un texto diferente y por tanto un CRC diferente.

La solución implementada consiste en calcular el CRC sobre el payload JSON sin el campo 'crc16', y adjuntar el CRC como campo adicional en el mensaje final. El receptor extrae el valor del campo 'crc16', elimina ese campo del documento JSON deserializado, re-serIALIZA el documento en el mismo formato canónico y calcula el CRC para compararlo. Dado que el receptor re-serIALIZA exactamente los mismos campos en el mismo orden (ArduinoJson preserva el orden de inserción), y dado que el emisor siguió el mismo orden al construir el objeto, los textos son idénticos y el CRC es reproducible.

El formato canónico de cada tipo de mensaje, utilizado como entrada al cálculo del CRC, es:

```

ESTADO_ACTUAL:
{"tipo":"ESTADO_ACTUAL","encendido":true,"color":"VERDE","r":0,"g":255,"b":0}

RESULTADO_CMD:
{"tipo":"RESULTADO_CMD","exito":true,"encendido":true,"color":"VERDE","r":0,"g":255,"b":0}

CMD_COLOR (D3 → D2):
{"tipo":"CMD_COLOR","color":"VERDE","r":0,"g":255,"b":0}

CMD_ENCENDER / CMD_APAGAR / CMD_DESCONECTAR:
{"tipo":"CMD_ENCENDER"}

```

La inclusión explícita de los valores r, g y b en el comando de ida garantiza que la firma CRC-16 proteja también los componentes numéricos del triplete de color contra corrupciones, sin depender únicamente de la cadena de texto del nombre simbólico.

7.2.2 Algoritmo CRC-16 — Implementación en C++ y JavaScript

La misma lógica de división polinómica se implementa en C++ (para D2) y en JavaScript (para D3). La clave de la verificación cruzada es que ambas implementaciones usen el mismo polinomio, el mismo valor inicial, el mismo orden de bits y las mismas operaciones.

Implementación en C++ (D2 — firmware ESP32):

```
uint16_t calcularCRC16(const char *payload, size_t len) {
    uint16_t crc = 0x0000;  // Init = 0x0000

    for (size_t i = 0; i < len; i++) {
        crc ^= (uint8_t)payload[i];

        for (uint8_t bit = 0; bit < 8; bit++) {
            if (crc & 0x0001) { // LSB-first
                crc = (crc >> 1) ^ CRC16_POLY; // 0x8005
            } else {
                crc >>= 1;
            }
        }
    }
    return crc;
}
```

Implementación en JavaScript (D3 — navegador):

```
function calcularCRC16(str) {
    const POLY = 0x8005;
    let crc = 0x0000;
    for (let i = 0; i < str.length; i++) {
        crc ^= str.charCodeAt(i);
        for (let bit = 0; bit < 8; bit++) {
            if (crc & 0x0001) {
                crc = ((crc >>> 1) ^ POLY) & 0xFFFF; // >>> evita signo
            } else {
                crc = (crc >>> 1) & 0xFFFF;
            }
        }
    }
    return crc;
}
```

La diferencia entre el operador >> y >>> en JavaScript es relevante: JS utiliza internamente el formato IEEE 754 de doble precisión para todos los números. El operador >> aplica desplazamiento aritmético (extiende el bit de signo), mientras que >>> aplica desplazamiento lógico sin signo (inserta ceros). Para simular un registro de

16 bits sin signo, se emplea >>> combinado con & 0xFFFF para descartar cualquier bit por encima de la posición 15. Sin estas precauciones, los valores con el bit 15 en 1 serían tratados como negativos y producirían resultados incorrectos.

7.2.3 Flujo de Verificación CRC-16: D3 → D2

El flujo completo de verificación cuando D3 envía un comando a D2 es el siguiente:

1. D3 construye el objeto JSON del comando sin el campo `crc16`.
2. D3 serializa el objeto a string con `JSON.stringify()` (formato canónico: claves en orden de inserción).
3. D3 calcula `calcularCRC16(stringSerializado)` y obtiene el valor de 16 bits.
4. D3 agrega el campo `crc16` al objeto y serializa nuevamente con `JSON.stringify()`.
5. D3 envía el string resultante por WebSocket.
6. D2 recibe el string en `onWsEvent()` → `procesarMensajeWS()`.
7. D2 parsea el JSON con `deserializeJson()` y extrae el valor del campo `crc16`.
8. D2 elimina el campo `crc16` del documento (`doc.remove("crc16")`).
9. D2 re-serializa el documento a string con `serializeJson()`.
10. D2 calcula `calcularCRC16(stringReserializado)` y compara con el valor recibido.
11. Si coinciden: el mensaje es válido, se procesa según el tipo. Si no coinciden: se responde con `CRC_ERROR` y se descarta.

7.2.4 Justificación de CRC-16 vs. CRC-8 para el Enlace WebSocket

La elección de CRC-16 para el enlace WebSocket en lugar del mismo CRC-8 empleado en el enlace IR responde a una razón técnica directa: la longitud de los mensajes JSON es sustancialmente mayor que la de las tramas IR.

Enlace	Longitud del mensaje	CRC elegido	HD garantizado	Capacidad de detección
IR (D1-D2)	3 bytes (24 bits de datos)	CRC-8/MAXIM	HD=4	Todos los errores de 1, 2 y 3 bits sobre 3 bytes
WebSocket (D2-D3)	Hasta ~300 bytes (2.400 bits)	CRC-16/IBM	HD=4	Todos los errores de 1, 2 y 3 bits sobre hasta 32.767 bits

Si se aplicara CRC-8/MAXIM al enlace WebSocket, la distancia de Hamming para mensajes de 2.400 bits se degradaría a HD=2, garantizando solo la detección de errores de un único bit. El CRC-16/IBM, en cambio, mantiene HD=4 para mensajes de hasta 32.767 bits (4.096 bytes), que supera ampliamente la longitud máxima de cualquier mensaje del sistema.

Adicionalmente, el CRC-16 tiene probabilidad de no detección de $1/2^{16} \approx 0.0015\%$ para errores de ráfaga que superen su capacidad, frente a $1/2^8 \approx 0.39\%$ del CRC-8. Esta diferencia es relevante cuando el medio de transmisión —en este caso Wi-Fi con el ruido propio de la banda de 2.4 GHz (hornos de microondas, Bluetooth, redes Wi-Fi

vecinas)— puede introducir ráfagas de errores que afecten múltiples bytes de un mensaje.

8. Persistencia de Estado — NVS / Preferences

El sistema implementa persistencia de estado en ambos dispositivos ESP32 (D1 y D2) mediante la biblioteca Preferences, que proporciona una abstracción de alto nivel sobre el sistema de almacenamiento No Volátil (NVS) del ESP32. NVS es un subsistema del framework ESP-IDF que gestiona el almacenamiento de pares clave-valor en la memoria flash interna del microcontrolador, implementando wear leveling automático para distribuir las escrituras uniformemente entre las páginas de flash y maximizar la vida útil del dispositivo.

La persistencia cumple el siguiente requisito de la consigna: 'El Dispositivo 2 debe ser capaz de conservar su estado si el Dispositivo 3 se desconecta'. Extensivamente, también D1 preserva su estado ante cortes de energía.

En D2, el estado persistido es mínimo: el índice del color actual en la tabla COLORES[] y el booleano encendido. Los valores RGB se reconstruyen desde la tabla al recuperar el índice, minimizando el número de escrituras en flash. En D1, se persisten directamente los tres bytes RGB, dado que D1 no mantiene el concepto de índice de paleta.

Dispositivo	Namespace NVS	Claves almacenadas	Contenido
D1	d1_state	magic, r, g, b	Centinela de validez + componentes RGB del último color
D2	d2_state	magic, color_idx, encendido	Centinela de validez + índice de paleta + estado on/off

El byte centinela (magic = 0xA5, patrón alternante 10100101b) es el mecanismo de detección de datos NVS inválidos o vacíos. Al recuperar el estado, el firmware verifica primero el centinela: si el valor leído coincide con NVS_MAGIC_VAL, los datos son válidos; si no coincide (NVS vacía, namespace nunca escrito, o corrupción), el sistema inicia con valores por defecto seguros. Este patrón alternante tiene una baja probabilidad de ocurrir aleatoriamente en flash no inicializada, reduciendo el riesgo de falsos positivos de validación.

9. Justificación de Decisiones de Diseño

9.1 Elección de Wi-Fi modo Ad Hoc (SoftAP) vs. Infraestructura

La comunicación Wi-Fi entre D2 y D3 podría haberse implementado en modo infraestructura (conectando ambos dispositivos a un router externo), pero se optó por el modo SoftAP por las siguientes razones:

- Independencia de infraestructura: el SoftAP no requiere router, switch ni ningún equipo de red preexistente. El sistema es auto-contenido y portable.
- Cumplimiento de consigna: la consigna especifica explícitamente que 'la conexión entre los dispositivos 2 y 3 no debe depender de Internet'. El SoftAP garantiza esto por diseño, ya que la red creada no tiene gateway ni DNS hacia Internet.
- Control de clientes: en modo SoftAP, D2 tiene visibilidad directa del número de clientes conectados y puede limitar el máximo a 1 (AP_MAX_CONN=1), garantizando que solo D3 accede al sistema.
- Dirección IP predecible: el AP del ESP32 siempre usa 192.168.4.1 como IP propia y asigna 192.168.4.2 al primer cliente por DHCP, eliminando la necesidad de descubrimiento de servicio.
- Seguridad: WPA2 protege el tráfico en el enlace inalámbrico, y la ausencia de gateway a Internet limita la superficie de ataque externa.

9.2 Elección de WebSocket vs. HTTP REST / MQTT / Bluetooth

Varias tecnologías fueron evaluadas para el enlace D2-D3:

Tecnología	Ventajas	Desventajas para este sistema
HTTP REST (polling)	Simple, sin estado en servidor	Latencia alta, no detecta desconexión, alto overhead por petición
MQTT	Ligero, pub/sub escalable	Requiere broker externo, no nativo en navegadores
Bluetooth SPP	Bajo consumo, sin Wi-Fi	Sin API nativa en navegadores; requiere app nativa
WebSocket (elegido)	Full-duplex, nativo en navegadores, detección de desconexión integrada	Overhead inicial de handshake HTTP (mínimo para uso continuo)

WebSocket fue la elección más adecuada por tres razones determinantes: la API WebSocket es nativa en todos los navegadores web modernos (no requiere extensiones ni aplicaciones adicionales en D3), el protocolo es full-duplex lo que permite a D2 notificar a D3 del resultado del ciclo IR sin que D3 haga polling, y el mecanismo Ping/Pong integrado del protocolo detecta automáticamente conexiones TCP 'zombie' que el nivel de transporte no descubriría de forma inmediata.

9.3 Elección de NEC Raw vs. Protocolo NEC Estándar

El protocolo NEC estándar invierte los bits de la dirección y el comando para incluir los bytes de complemento como mecanismo de verificación de integridad. Esta inversión es incompatible con el uso libre de los 32 bits de la trama para empaquetar R, G, B y CRC, ya que los bytes 2 y 4 estarían forzados a ser los complementos de los bytes 1 y 3.

La variante NEC Raw (sendNECRaw / decodedRawData en la biblioteca IRremote) preserva los 32 bits exactamente como los construye la aplicación, sin inversiones ni complementos. Esto permite la estructura libre [R][G][B][CRC] y es la razón técnica de la elección de esta variante.

9.4 CRC por Software vs. Uso de Bibliotecas Externas

En el contexto académico del proyecto, se optó deliberadamente por implementar el CRC mediante código propio (algoritmo bit a bit, aproximadamente 15 líneas por función) en lugar de usar bibliotecas como FastCRC o CRCLib. Esta decisión tiene fundamento técnico y pedagógico:

- **Transparencia:** la implementación propia hace explícita cada paso del algoritmo, facilitando la comprensión y la defensa oral del trabajo.
- **Portabilidad:** el mismo algoritmo se implementa con cambios mínimos en C++ (D2) y JavaScript (D3), sin dependencia de bibliotecas específicas de plataforma.
- **Controlabilidad:** el valor de inicialización, el polinomio y el orden de bits son directamente visibles y modificables en el código.
- **Tamaño:** 15 líneas de código vs. una dependencia de biblioteca que puede incluir funciones innecesarias para el proyecto.

9.5 Modelo de Concurrencia: Dual-Core con Variables Volatile

La alternativa de usar un mutex (portMUX_TYPE o SemaphoreHandle_t de FreeRTOS) fue evaluada y descartada por innecesaria para el patrón de acceso del sistema. El análisis del código muestra un único escritor por cada variable compartida: el callback WebSocket (Core 0) solo escribe irPendiente=true y los valores pendientes; el loop (Core 1) solo los lee y limpia. En este patrón de single-writer, la declaración volatile es suficiente para garantizar la visibilidad entre cores en el ESP32 (arquitectura coherente de caché), sin el overhead de un mutex.

Si en el futuro el sistema requiriera escritura concurrente desde ambos cores sobre las mismas variables, la migración a un mutex sería necesaria. La documentación del código incluye esta condición explícitamente para facilitar el mantenimiento.

10. Problemas Encontrados y Soluciones Implementadas

10.1 Indeterminismo Estructural en el Cálculo de CRC-16 sobre JSON

Durante el desarrollo inicial de la verificación CRC-16 en el enlace WebSocket, el campo 'crc16' se incrustaba directamente como una clave más dentro del objeto JSON. En el receptor, se parseaba el documento, se eliminaba el campo 'crc16' y se re-serIALIZABA para calcular el CRC de verificación. Este enfoque falló en pruebas porque el estándar JSON (ECMA-404) no garantiza el orden de las claves en la representación serializada. Al deserializar y re-serIALIZAR el documento en el ESP32 con ArduinoJson, las claves podían aparecer en orden diferente al original, produciendo un string diferente y un CRC de verificación que no coincidía con el transmitido, rechazando mensajes legítimos.

Solución implementada: el Patrón de 'Sobre' (Envelope Pattern). El payload se serializa primero a un string inmutable sin el campo crc16. Se calcula el CRC sobre ese string exacto. Luego, se agrega el campo crc16 al objeto y se realiza la serialización final. El receptor invierte el proceso: extrae el CRC, elimina el campo del documento, re-serIALIZA y compara. Dado que ArduinoJson preserva el orden de inserción de claves, y dado que tanto el emisor como el receptor construyen el documento en el mismo orden, el string re-serIALIZADO es idéntico al original, garantizando la reproducibilidad del CRC.

10.2 Bucle Infinito por 'Eco Óptico' en el Enlace IR con CRC Init=0x00

Durante las pruebas del enlace IR con CRC-8, se detectó un comportamiento anómalo: tras la confirmación exitosa de un color, D1 y D2 comenzaban a intercambiar mensajes de forma continua, saturando el canal IR. El análisis del problema reveló la causa raíz: al emitir el ACK IR, D1 reactivaba el receptor KY-022 casi instantáneamente (mediante IrReceiver.start()). El receptor captaba reflexiones de la propia transmisión ACK en el entorno físico (reflexiones en paredes, objetos, la propia placa), que la biblioteca IRremote interpretaba como una trama. El valor de esa trama espuria era 0x00000000 (todos los bits en cero).

Con el valor de inicialización del registro CRC en 0x00, el cálculo del CRC-8 sobre tres bytes de valor cero producía CRC=0x00. La trama espuria 0x00000000 tiene como byte de CRC el valor 0x00, que coincidía con el CRC calculado por el receptor, resultando en una validación falsa. D1 interpretaba el ruido como un comando válido 'Apagar' (RGB=0,0,0), emitía otro ACK, que a su vez generaba otro eco, estableciendo el bucle.

Solución implementada: cambiar el valor de inicialización del registro CRC de 0x00 a 0xFF (Init=0xFF, valor estándar de CRC-8/MAXIM). Con este cambio, el CRC de tres bytes nulos es un valor no nulo, y la trama espuria 0x00000000 ya no pasa la validación de CRC, eliminando el bucle.

10.3 Latencia en la Actualización de LEDs de Estado ante Desconexión Voluntaria

Al implementar el botón de desconexión en D3, se observó que los LEDs de estado en D2 (LED_CONN y LED_DISC) no se actualizaban de forma inmediata al presionar el botón. El tiempo de latencia podía ser de varios segundos. La causa raíz fue identificada en el comportamiento de la pila TCP asíncrona (AsyncTCP / lwIP): cuando D2 ejecutaba

client->close()) unilateralmente para cerrar el socket, el handshake TCP de finalización (secuencia FIN/ACK de cuatro vías) quedaba pendiente en el buffer de la pila lwIP. El evento WS_EVT_DISCONNECT en el callback de ESPAsyncWebServer se disparaba solo cuando la pila completaba la secuencia, lo que podía demorar varios segundos según el estado de los buffers.

Solución implementada: protocolo de 'Cierre Negociado' a nivel de aplicación. D3 envía CMD_DESCONECTAR (con CRC-16 válido). D2 lo recibe, valida el CRC, responde con ACK_DESCONECTAR (con CRC-16) y no cierra el socket. D3 recibe el ACK y ejecuta ws.close(1000, 'Cierre voluntario'), cerrando el socket desde el cliente. Esta secuencia garantiza que el cierre TCP es iniciado por el cliente, lo que permite al motor asíncrono del ESP32 completar el handshake TCP limpiamente y disparar WS_EVT_DISCONNECT con mínima latencia.

10.4 Conflicto de Canal LEDC 0 con IRremote

Durante el desarrollo inicial, la asignación del canal LEDC 0 al canal rojo del LED RGB producía comportamientos erróneos: la transmisión IR fallaba con patrones aleatorios, o el brillo del LED rojo fluctuaba inexplicablemente. La investigación del código fuente de IRremote reveló que la biblioteca reserva internamente el canal LEDC 0 (y el canal RMT 0) para generar la portadora de 38 kHz del emisor IR. La colisión de ambas configuraciones sobre el mismo canal producía el comportamiento observado. La solución fue asignar el canal rojo del LED al LEDC_CH_R=3, desplazándolo fuera del rango utilizado por IRremote.

10.5 Falso Positivo por Auto-Acuse de Recibo (Sordera por Eco)

Durante las pruebas, se detectó que el Maestro (D2) confirmaba falsamente la recepción de comandos, aunque el Esclavo (D1) no estuviese en línea directa de emisores-receptores. La causa raíz radicaba en la simetría estructural del protocolo y el esquema half-duplex del hardware. El emisor IR (KY-005) y receptor IR (KY-022) de D2 se encuentran físicamente adyacentes; al transmitir un comando y reactivar el receptor, D2 capturaba el rebote óptico de su propia transmisión en el ambiente.

Al ser una trama legítima, su CRC era matemáticamente válido, engañando a la validación. La solución implementada consistió en introducir Asimetría de Protocolo: D1 fue reprogramado para responder invirtiendo bit a bit su firma (~CRC). D2 fue reprogramado para esperar exclusivamente el complemento matemático, descartando inmediatamente los ecos propios que porten la firma original.

11. Flujo de Datos Extremo a Extremo

11.1 Secuencia de Conexión Inicial de D3

Cuando el usuario abre el navegador y navega a <http://192.168.4.1>:

1. El navegador establece una conexión TCP al puerto 80 del AP (D2).
2. El navegador realiza el upgrade HTTP → WebSocket (handshake 101 Switching Protocols).
3. ESPAsyncWebServer dispara WS_EVT_CONNECT en el Core 0 de D2.

4. D2 actualiza los LEDs: LED_CONN encendido, LED_DISC apagado.
5. D2 construye el mensaje ESTADO_ACTUAL con el estado actual del sistema, calcula CRC-16 y lo envía al cliente.
6. D3 recibe ESTADO_ACTUAL, valida el CRC-16 y actualiza la interfaz gráfica con el estado real del LED de D1.

11.2 Secuencia de Cambio de Color

Cuando el usuario presiona un botón de color en D3:

7. D3 construye el objeto CMD_COLOR con el nombre del color y sus componentes numéricos (R, G, B).
8. D3 serializa el objeto a string, calcula CRC-16 y agrega el campo crc16.
9. D3 envía el string por WebSocket a D2.
10. D2 recibe el frame en WS_EVT_DATA → procesarMensajeWS().
11. D2 extrae crc16, elimina el campo, re-serializa y verifica el CRC. Si el CRC es inválido, responde con CRC_ERROR y termina.
12. D2 identifica el tipo CMD_COLOR, busca el índice del color en la tabla COLORES[], carga los valores RGB en las variables pendientes y activa irPendiente=true.
13. El loop() del Core 1 detecta irPendiente=true y llama a ejecutarCicloIR().
14. ejecutarCicloIR() llama a enviarColorIR(r, g, b): calcula CRC-8, construye la trama de 32 bits, detiene el receptor, transmite la trama NEC Raw, reactiva el receptor y espera el ACK por hasta 2000 ms.
15. D1 recibe la trama: extrae R, G, B, CRC. Calcula CRC-8 sobre R, G, B. Si coincide, aplica el color al LED, guarda en NVS y transmite el ACK con los mismos datos e invirtiendo la firma (~CRC).
16. D2 recibe el ACK: extrae R, G, B, CRC del ACK. Verifica que CRC-8 corresponda al CRC asimétrico (~CRC) y coincidencia de valores. Si es válido, actualiza el estado, aplica el color al LED de D2 y guarda en NVS.
17. D2 llama a notificarD3Estado(true): construye RESULTADO_CMD con exito=true, calcula CRC-16, envía broadcast a todos los clientes WebSocket.
18. D3 recibe RESULTADO_CMD, valida CRC-16, actualiza la interfaz (preview de color, valores RGB numéricos, resaltado del botón activo) y muestra el banner de confirmación.

12. Verificación y Pruebas del Sistema

12.1 Casos de Prueba Funcionales

ID	Caso de prueba	Procedimiento	Resultado esperado	Resultado obtenido
TC-01	Cambio de color exitoso	Seleccionar VERDE en D3	LED D1 en verde, RESULTADO_CMD exito=true, LED D2 en verde	OK
TC-02	Apagado del LED	Presionar APAGAR en D3	LED D1 apagado (RGB=0,0,0), RESULTADO_CMD exito=true	OK
TC-03	Reencendido con último color	Presionar ENCENDER en D3 tras apagado	LED D1 en el último color confirmado	OK
TC-04	Desconexión voluntaria D3	Presionar DESCONECTAR en D3	LED DISC de D2 encendido, LED D1 sin cambios, badge D3 = Desconectado	OK
TC-05	Reconexión de D3	Presionar RECONECTAR en D3	D3 recibe ESTADO_ACTUAL, interfaz muestra el estado real	OK
TC-06	Persistencia al corte de energía D2	Apagar y encender D2	D2 recupera el estado desde NVS y aplica el color al LED D2	OK
TC-07	Persistencia al corte de energía D1	Apagar y encender D1	D1 recupera el color desde NVS y lo aplica al LED D1	OK
TC-08	Detección de CRC-16 inválido	Modificar el campo crc16 en un mensaje de prueba	D2 responde con CRC_ERROR, mensaje descartado, D3 muestra alerta	OK
TC-09	Detección de CRC-8 inválido en IR	Simular ruido en el canal IR	D1 descarta la trama, no cambia el LED, no envía ACK	OK
TC-10	Reintentos IR ante fallo de ACK	Bloquear el canal IR durante el envío	D2 reintentará 3 veces y notifica fallo a D3	OK
TC-11	Detección de desconexión abrupta de D3	Cerrar el navegador sin presionar DESCONECTAR	D2 detecta la pérdida de heartbeat y activa LED DISC	OK

12.2 Análisis de Señales — Osciloscopio

13. Conclusiones

El sistema de control domótico RGB desarrollado en el marco del TP3 Integrador cumple la totalidad de los requisitos de la consigna y supera las capacidades del TP2 en los siguientes aspectos técnicos:

- Verificación de integridad de datos: la incorporación de CRC-8/MAXIM en el enlace IR y CRC-16/IBM en el enlace WebSocket añade una capa matemática de detección de errores que el TP2 no poseía. La elección de los polinomios está justificada técnicamente por la longitud de los mensajes en cada enlace y la distancia de Hamming requerida.
- Robustez ante escenarios adversos: el sistema maneja correctamente el eco óptico, la corrupción de tramas, la pérdida de conectividad Wi-Fi y los cortes de energía, recuperando el estado correcto en todos los casos.
- Desconexión controlada: el protocolo de cierre negociado `CMD_DESCONECTAR` / `ACK_DESCONECTAR` garantiza la actualización inmediata de los indicadores físicos de D2 sin latencias propias del cierre TCP asíncrono.
- Interfaz gráfica funcional: la interfaz web embebida en el ESP32 opera sin dependencias externas, siendo compatible con cualquier dispositivo con navegador moderno, sin instalación de software adicional en D3.
- Arquitectura dual-core: la separación de la lógica de red (Core 0) y la lógica IR (Core 1) permite que el sistema continúe detectando eventos de red durante los períodos de transmisión IR, que pueden durar hasta ~6.9 segundos en el peor caso.

Desde el punto de vista académico, el proyecto permitió integrar en una única implementación funcional los conceptos fundamentales de la comunicación de datos estudiados durante el cuatrimestre: modulación de portadora (IR a 38 kHz), protocolos de enlace (NEC Raw), redes de área local inalámbricas (IEEE 802.11 / WPA2), protocolos de aplicación (HTTP/WebSocket), serialización de datos (JSON), verificación de integridad (CRC), gestión de concurrencia (FreeRTOS dual-core) y persistencia de datos en sistemas embebidos (NVS).

13.1 Propuestas de Mejora y Escalabilidad

- Tablas de búsqueda para CRC (lookup tables): la implementación bit a bit del CRC puede reemplazarse por tablas precomputadas de 256 entradas, reduciendo el tiempo de cálculo de $O(8 \times n)$ a $O(n)$ operaciones simples de XOR. Para los volúmenes de datos del sistema actual, la diferencia es imperceptible, pero sería relevante en aplicaciones con mayor throughput.
- Soporte de múltiples clientes D3: el Access Point actual está configurado para máximo 1 cliente (`AP_MAX_CONN=1`). Extender el sistema a múltiples clientes simultáneos requeriría sincronización adicional entre callbacks y la lógica de aplicación.
- Cifrado adicional en la capa de aplicación: el tráfico WebSocket es cifrado por WPA2 en la capa Wi-Fi, pero no emplea TLS (WSS). En un escenario de despliegue productivo, WSS (WebSocket Secure) añadiría cifrado de extremo a extremo independiente de la capa de enlace.

- Control de color libre (selector de color): la consigna menciona la posibilidad de elegir cualquier color RGB. La paleta actual de 12 colores predefinidos podría complementarse con un selector de color tipo 'color picker' HTML5 para control libre de los tres componentes R, G, B.
- CRC con tabla de búsqueda en flash: en el ESP32, las tablas de lookup pueden almacenarse en flash (DROM) para no ocupar SRAM. Esta optimización combina la velocidad de las tablas con el uso eficiente de la memoria.

14. Referencias Bibliográficas

Espressif Systems. (2024). ESP32 Technical Reference Manual (versión 5.4). Espressif Systems.

https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

Espressif Systems. (2024). ESP-IDF Programming Guide: Non-Volatile Storage Library.

https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/storage/nvs_flash.html

Fette, I., & Melnikov, A. (2011). The WebSocket Protocol (RFC 6455). Internet Engineering Task Force (IETF). <https://datatracker.ietf.org/doc/html/rfc6455>

IEEE Standard for Information Technology—Telecommunications and Information Exchange Between Systems Local and Metropolitan Area Networks—Specific Requirements Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications, IEEE Std 802.11-2020, 2021. <https://doi.org/10.1109/IEEESTD.2021.9363693>

Koopman, P. (2002). 32-Bit Cyclic Redundancy Codes for Internet Applications. Proceedings of the International Conference on Dependable Systems and Networks. <https://doi.org/10.1109/DSN.2002.1028931>

Stallings, W. (2017). Data and Computer Communications (10.^a ed.). Pearson Education.

IRremote Library Contributors. (2024). IRremote Arduino Library (versión 4.7.1) [Código fuente]. GitHub. <https://github.com/Arduino-IRremote/Arduino-IRremote>

Blanchon, B. (2024). ArduinoJson (versión 7.2.0) [Biblioteca de software]. <https://arduinojson.org/>

PlatformIO. (2024). PlatformIO IDE Documentation. <https://docs.platformio.org/>

me-no-dev. (2024). ESPAsyncWebServer [Código fuente]. GitHub. <https://github.com/me-no-dev/ESPAsyncWebServer>

15. Repositorio GitHub y contenido multimedia.

Enlace al repositorio GitHub con los firmwares de cada dispositivo:

<https://github.com/bevilacquafrancisco/Comunicacion-de-Datos>

Enlace a video de demostración práctica funcional del sistema:

https://drive.google.com/file/d/18nojuSqkJLDHJZeYuLfuCWLGzYffFRQQ/view?usp=drive_link